

ΥΠΟΛΟΓΙΣΤΙΚΗ ΝΟΗΜΟΣΥΝΗ

ΕΡΓΑΣΙΑ 1: ΓΕΝΕΤΙΚΟΙ ΑΛΓΟΡΙΘΜΟΙ



ΔΗΜΟΚΡΙΤΕΙΟ
ΠΑΝΕΠΙΣΤΗΜΙΟ
ΘΡΑΚΗΣ

ΤΜΗΜΑ
ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ
& ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ

Ιατρού Κωνσταντίνος 58071

Άσκηση 1 (Knapsack Problem)

ΠΙΝΑΚΑΣ ΠΕΡΙΕΧΟΜΕΝΩΝ

Α. Εισαγωγή.....	2
Β. Αλγόριθμος Επίλυσης και επί μέρους Συνρτήσεις.....	3
Γ. Σημειώσεις.....	18

A. ΕΙΣΑΓΩΓΗ

Το πρόβλημα που καλούμαι να επιλύσω είναι να χωρέσω σε ένα σακίδιο αντικείμενα με την μέγιστη δυνατή αξία, με βασική προϋπόθεση να χωρέσουν στο σακίδιο.

Εφόσον δίνεται η λίστα με τα 100 διαφορετικά αντικείμενα, μπορούμε να αναπαραστήσουμε τις λύσεις ως ένα διάνυσμα μήκους 100 θέσεων με δυαδικά ψηφία σε κάθε θέση. Αν στην θέση i του διανύσματος υπάρχει η λογική τιμή 1 τότε το αντικείμενο στη θέση i , της λίστας που δόθηκε, περιέχεται στην προτεινόμενη λύση, εναλλακτικά αν στο διάνυσμα στη θέση i έχει την λογική τιμή 0 τότε το αντίστοιχο αντικείμενο στην λίστα που δόθηκε δεν περιέχεται στην προτεινόμενη λύση.

Δίνεται ένα μικρότερο παράδειγμα για την επεξήγηση της μεθοδολογίας που θα χρησιμοποιηθεί για την επίλυση του προβλήματος.

Για παράδειγμα έστω ότι η λίστα που δίνεται είναι μήκους 5 και περιέχει τα παρακάτω αντικείμενα:

ID	Size	Value
1	4	10
2	1	2
3	6	17
4	2	3
5	4	5

Τότε η αναπαράσταση μίας από της προτεινόμενες λύσεις αν είναι η παρακάτω:

[0 1 0 1 0]

σημαίνει ότι στην προτεινόμενη λύση τα περιεχόμενα του σακιδίου είναι τα αντικείμενα με τα ID 2 και 4.

Με αυτό τον τρόπο έχουμε μία αναπαράσταση που ταιριάζει στην θεωρία που παραδόθηκε στο 1^ο εργαστήριο για τους γενετικούς αλγορίθμους και θα είναι εύκολο να εφαρμοστούν οι μεθοδολογίες που διδάχθηκαν όσον αφορά την αναπαραγωγή, την μετάλλαξη κλπ.

Όλες οι συναρτήσεις γράφτηκαν έτσι ώστε, όσο είναι δυνατόν, να μπορούν να χρησιμοποιηθούν και για knapsack πρόβλημα με διαφορετικά χαρακτηριστικά, και αριθμό διαθέσιμων αντικειμένων.

Για το παρόν πρόβλημα οι συναρτήσεις γράφτηκαν και εκτελέστηκαν επιτυχώς στις εκδόσεις του Matlab R2020b και R2021a.

Β. ΑΛΓΟΡΙΘΜΟΣ ΕΠΙΛΥΣΗΣ ΚΑΙ ΕΠΙ ΜΕΡΟΥΣ ΣΥΝΑΡΤΗΣΕΙΣ

1. Εισαγωγή Δεδομένων

Για την εισαγωγή των δεδομένων από το αρχείο knapsack.xls που δόθηκε, δημιουργήθηκε η συνάρτηση imp_vals(filename) και δίνεται παρακάτω:

```

1  function [sOfItems, vOfItems, solSize, weighTarget, maxItems] = imp_vals(filename)
2  -     if nargin<1
3  -         filename = 'knapsack.xls';
4  -     end
5  -     %opts = detectImportOptions('knapsack.xls', 'DataRange', 'A3:C102');
6  -
7  -     sOfItems = xlsread('knapsack.xls','B:B');
8  -     vOfItems = xlsread('knapsack.xls','C:C');
9  -     weighTarget = readmatrix(filename, 'Range', 'A1:A1');
10 -     solSize = readmatrix(filename, 'Range', 'A2:A2');
11 -
12 -     sortedSize = sort(sOfItems);
13 -     maxItems = 0;
14 -     temp = 0;
15 -     i = 1;
16 -
17 -     while temp<weighTarget
18 -         if (temp+sortedSize(i))<weighTarget
19 -             maxItems = maxItems + 1;
20 -         end
21 -         temp = temp + sortedSize(i);
22 -         i = i + 1;
23 -     end
24 - end

```

Οι έξοδοι της συνάρτησης είναι οι πίνακες sOfItems και vOfItems, και οι τιμές που περιέχονται στις μεταβλητές solSize, weighTarget και maxItems.

Οι τιμές των εξόδων δίνονται ως εξής:

Στην γραμμή 7 εκχωρούμε στην μεταβλητή sOfItems τα μεγέθη κάθε αντικειμένου σε μορφή λίστας, ενώ ομοίως στην σειρά 8 εκχωρούμε σε μορφή λίστας στην μεταβλητή vOfItems την αξία του κάθε αντικειμένου. Οι δύο αυτές εντολές είναι έτσι δομημένες έτσι ώστε να εισάγουν ολόκληρες τις λίστες από excel αρχείο χωρίς να λαμβάνουν υπόψη το μέγεθος τους. Έτσι αν κάποιος έχει ένα διαφορετικό σετ αντικειμένων, αν τα εισάγει στο αρχείο διατηρώντας την μορφή, μπορεί να λύσει και το εκείνο το πρόβλημα.

Στην γραμμή 9 παίρνουμε από το αρχείο, που δίνεται από τον χρήστη, το μέγιστο συνολικό μέγεθος των αντικειμένων που χωράει το σακίδιο που έχουμε.

Στην γραμμή 10 έχουμε τον αριθμό των συνολικών αντικειμένων από τα οποία μπορούμε να επιλέξουμε για να τοποθετήσουμε στο σακίδιο.

Στην συνέχεια ταξινομούμε τα μεγέθη που υπάρχουν στην λίστα με τα μεγέθη των αντικειμένων σε αύξουσα σειρά και εκχωρούμε την νέα ταξινομημένη λίστα στην μεταβλητή `sortedSize`.

Μέσα από την `while loop` χρησιμοποιούμε την ταξινομημένη λίστα παίρνοντας τα μικρότερα αντικείμενα και με αυτόν τον τρόπο υπολογίζουμε πόσα θα είναι τα περισσότερα δυνατά αντικείμενα που μπορούμε να τοποθετήσουμε μέσα στο σακίδιο. Εν τέλει εκχωρούμε αυτή την μέγιστη τιμή στην μεταβλητή `maxItems`.

Αυτό το κάνουμε για να επιταχύνουμε την διαδικασία. Έτσι ο αλγόριθμος θα αποφύγει να δημιουργήσει ακραίους συνδυασμούς που δεν υπάρχει περίπτωση να χωρέσουν στο σακίδιο.

Θα φανεί αυτό με την εκτέλεση της συνάρτησης στο παρακάτω παράδειγμα:

```
>>[a b c d e] = imp_vals()
```

Οι μεταβλητές που παίρνουμε είναι οι εξής:

Workspace		
Name ▲	Value	
a	100x1 double	
b	100x1 double	
c	100	
d	27	
e	3	

Η μεταβλητή `a` όπως είπαμε πριν περιέχει τα μεγέθη για κάθε αντικείμενο και `b` τις αξίες.

Η μεταβλητή `c` περιέχει την τιμή 100 που είναι τα συνολικά διαφορετικά αντικείμενα τα οποία μπορούμε να επιλέξουμε.

Η `d` περιέχει την τιμή 27 που είναι το άθροισμα των μεγεθών που χωράει το σακίδιο.

Η `e` περιέχει την τιμή 3 που είναι μέγιστος συνδυασμός αντικειμένων που μπορούμε να έχουμε. Γιατί;

Γιατί αν δούμε τα περιεχόμενα τις λίστες με τα μεγέθη τα 5 μικρότερα αντικείμενα έχουν μεγέθη 7,7,7,7 και 8. Συνεπώς χρησιμοποιώντας μόνο τα μικρότερα σε μέγεθος αντικείμενα μπορούμε να πάρουμε το πολύ 3 αντικείμενο των 7. Συνεπώς δεν υπάρχει νόημα να έχουμε προτεινόμενες λύσεις με 4 ή περισσότερα αντικείμενα επιλεγμένα. Με αυτόν τον τρόπο περιορίζουμε σε τεράστιο βαθμό το σύνολο των πιθανών λύσεων στο οποίο θα εργαστούμε. Ποιο συγκεκριμένα θα έπρεπε να υπολογίσουμε το παρακάτω άθροισμα για να δούμε ποιος είναι ο συνολικός αριθμός συνδυασμών που υπάρχουν:

$$\sum_{i=1}^{100} \binom{100}{i}$$

Το άθροισμα αυτό είναι τόσο μεγάλο που δεν μπορεί να αναπαρασταθεί ούτε στο MATLAB.

Ακόμη, αξίζει να αναφερθεί ότι ο χρήστης έχει την δυνατότητα να εισάγει δικό του αρχείο από το οποίο η συνάρτηση θα εισάγει τα χαρακτηριστικά του knapsack problem, αλλά αν δεν το κάνει παίρνει τα στοιχεία σαν default επιλογή από το αρχείο knapsack.xls

Σημείωση: Μετά από δοκιμές η εκτέλεση της εν λόγω συναρτήσεως είναι αρκετά ακριβή υπολογιστικά σε χρόνο, οπότε τις εξόδους της αποφάσισα να τις χρησιμοποιήσω ως εισόδους σε άλλες συναρτήσεις για να μην χρειαστεί να την εκτελώ πολλαπλές φορές σε μικρές συναρτήσεις που τρέχουν εσωτερικά σε άλλες συναρτήσεις. Συναρτήσεις όπως η αυτή του υπολογισμού της ευρωστίας και αυτή της επιλογή των γονέων εκτελούνται εκατοντάδες ή και χιλιάδες φορές κατά την διάρκεια της προσπάθειας επίλυσης του προβλήματος. Οπότε η `imp_vals` καλείται ελάχιστες φορές και κυρίως από τις «ανώτερες» συναρτήσεις.

2. Παραγωγή αρχικού πληθυσμού

Για την παραγωγή του αρχικού πληθυσμού δημιουργήθηκε η παρακάτω συνάρτηση με όνομα `gen_init_population(count=10)`, δηλαδή αν δεν δοθεί τιμή ως argument θα θεωρείται από default ότι το μέγεθος του πληθυσμού θα είναι 10:

```

1  function a = gen_init_population(count, solSize, maxItems)
2  -     if nargin<3
3  -         [~, ~, solSize, ~, maxItems] = imp_vals();
4  -     end
5
6
7
8  -     population = nan(count, solSize);
9
10 -     sizeP = size(population);
11 -     for i=1:count
12 -         bits = zeros(1,solSize);
13 -         numOnes = randi([1,maxItems]);
14 -         p = int32(randperm(solSize, numOnes));
15 -         for j=1:length(p)
16 -             bits(p(j)) = 1;
17 -         end
18
19 -         population(i, 1:sizeP(2)) = bits;
20 -     end
21
22 -     a = population;
23 - end

```

Η default τιμή για τα `solSize` και `maxItems` ορίζεται στις γραμμές 2-4. Η `solSize` θα δίνει πόσο μήκος θα έχει η κάθε υποψήφια λύση και το `maxItems` πόσα αντικείμενα, κατά μέγιστο, μπορεί να περιέχονται σε μία υποψήφια λύση.

Στην γραμμή 8 αρχικοποιείται ο πίνακας που θα εισάγουμε τον αρχικό πληθυσμό ώστε να έχει τις διαστάσεις του πληθυσμού αλλά με NaN αρχικές τιμές, τις οποίες θα αλλάξουμε στην συνέχεια.

Στην συνέχεια μέσω του `for loop` εισάγουμε σε κάθε σειρά του πίνακα `population` μία από τις αρχικές προτεινόμενες λύσεις.

Ο τρόπος με τον οποίο λειτουργεί ο βρόχος είναι ότι, αρχικά, στην γραμμή 12 δημιουργείται ένα διάνυσμα με μηδενικά με μήκος `solSize`.

Στην συνέχεια παράγουμε ένα «τυχαίο» αριθμό από το 1 μέχρι το `maxSize` και τον εκχωρούμε στην μεταβλητή `numOnes`. Ο αριθμός αυτός, στην ουσία, είναι ο αριθμός των αντικειμένων που θα περιέχεται στην συγκεκριμένη προτεινόμενη λύση που παράγεται σε αυτό τον γύρο της επανάληψης

Στην γραμμή 14 παράγουμε τυχαίους αριθμούς από το 1 μέχρι το μέγεθος που μπορεί να έχει κάθε προτεινόμενη λύση. Οι τιμές που θα παραχθούν θα είναι τόσες όσο περιέχεται στην μεταβλητή `numOnes` και δεν θα υπάρχουν διπλές τιμές. Αυτές οι τιμές εκχωρούνται σε μορφή διανύσματος στην μεταβλητή `p`.

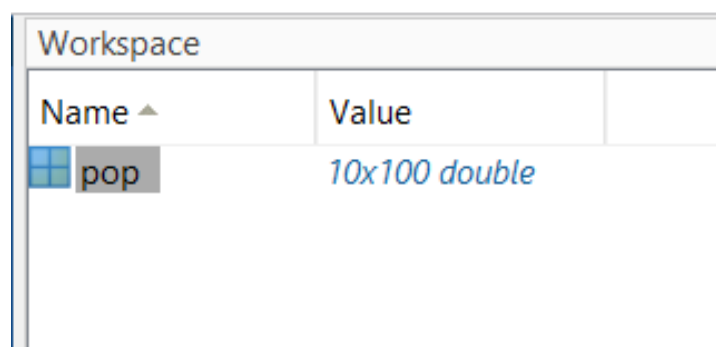
Στην συνέχεια έχουμε μία `for loop`, η οποία πηγαίνει στις θέσεις της προτεινόμενης λύσης (μεταβλητή `bits`), που δίνονται από το διάνυσμα `p`, και μετατρέπει το 0 σε 1.

Τέλος η προτεινόμενη λύση που περιέχεται στη μεταβλητή `bits` προσαρτάται στον πίνακα `population` και συνεχίζει στην επόμενη επανάληψη, αν δεν έχει παραχθεί όλος ο αρχικός πληθυσμός.

Παράδειγμα εκτέλεσης της συνάρτησης:

```
>> pop = gen_init_population(10);
```

Στην μεταβλητή `pop` εισάγουμε στην ουσία τον αρχικό πληθυσμό που παράγει η συνάρτηση. Το μέγεθος της μεταβλητής `pop` φαίνεται παρακάτω:



Workspace	
Name	Value
pop	10x100 double

Ενώ τα περιεχόμενα έχουν την παρακάτω μορφή:

	1	2	3	4	5	6	7	8	9	10	11	12
1	0	0	0	0	0	0	0	0	0	0	0	0
2	0	0	0	1	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0	0	0
5	0	0	0	0	0	0	0	0	0	0	0	0
6	0	0	0	0	0	0	0	0	0	0	0	0
7	0	0	0	0	0	0	0	0	0	0	0	1
8	0	0	0	0	0	0	0	0	0	0	0	0
9	0	0	0	0	0	0	0	0	0	0	0	0
10	0	0	0	0	0	0	0	0	0	0	0	0
11												

3. Fitness Coefficient

Για των υπολογισμό της ευρωστίας κάθε λύσης γράφηκε μία συνάρτηση η οποία υπολογίζει το άθροισμα των αξιών των αντικειμένων που επιλέγονται σε κάθε υποψήφια λύση.

Αν το βάρος των επιλεγμένων αντικειμένων, όμως, είναι μεγαλύτερο του επιθυμητού ορίου που ορίζεται μέσα από το αρχείο, τότε το αποτέλεσμα της συνάρτησης γίνεται 0 ανεξάρτητα από τις αξίες των αντικειμένων που επιλέχθηκαν.

Ο κώδικας της συνάρτησης είναι ο παρακάτω:

```

1 function fitness = fit_co(population, sizes_list, values_list, solSize, maxSize)
2     if nargin<5
3         [sizes_list, values_list, solSize, maxSize, ~] = imp_vals();
4     end
5
6     fitness = nan(size(population, 1), 1);
7
8     sizeP = size(population);
9
10    for i=1:sizeP(1)
11        tot_val = 0;
12        tot_weight = 0;
13
14
15        for j=1:solSize
16            tot_val = tot_val + values_list(j)*population(i,j);
17            tot_weight = tot_weight + sizes_list(j)*population(i,j);
18        end
19
20        if tot_weight>maxSize
21            tot_val = 0;
22        end
23
24        fitness(i) = tot_val;
25    end
26
27 end

```


4. Crossover

Αυτή η συνάρτηση, με όνομα `crossover(parent1, parent2, soLen)` εκτελεί την αναπαραγωγή, βλέποντας σε ποιες θέσεις έχει '1' ο κάθε γονιός και συνδυάζοντάς του με τον παρακάτω τρόπο:

Αν και οι 2 γονείς έχουν από ένα '1' τότε «κόβει» το διάνυσμα με τις θέσεις των '1' και τα συνδυάζει χιαστί.

Δηλαδή, αν `parent1 = [ 21, 56, 78]` και `parent2 = [ 16, 93]` και μέσω μιας ψευδοτυχαίας συνάρτησης δίνει σε μία μεταβλητή `a` μία τιμή από 1 μέχρι `min(length(parent1), length(parent2))`. Έστω η τιμή αυτή είναι ίση με 1. Άρα, θα μπορούσε να αναπαρασταθεί και ως εξής:

Οι απόγονοι θα είναι:

`parent1 = [ 21 | 56, 78]`

`child1 = [16, 93]`

`parent2 = [ 16 | 93]`

`child2 = [21, 56, 78]`

Ο κώδικας της συνάρτησης είναι ο παρακάτω:

```

1  function [child1, child2] = crossover(parent1, parent2, soLen)
2  -     if nargin<3
3  -         [~, ~, soLen, ~, ~] = imp_vals();
4  -     end
5  -     parent1_Ones = find(parent1==1);
6  -     parent2_Ones = find(parent2==1);
7
8  -     if length(parent1_Ones)>1 && length(parent2_Ones)>1
9  -         a = randi([1, min(length(parent1_Ones), length(parent2_Ones))]);
10
11 -         child1 = zeros(1, soLen);
12 -         child2 = zeros(1, soLen);
13
14 -         child1(parent2_Ones(1:a)) = 1;
15 -         child2(parent1_Ones(1:a)) = 1;
16
17 -         child1(parent1_Ones(a+1:end)) = 1;
18 -         child2(parent2_Ones(a+1:end)) = 1;
19
20 -     else
21 -         child1 = parent1;
22 -         child2 = parent2;
23 -     end
24 - end

```

Επειδή δεν είναι εύκολο να φανούν τα αποτελέσματα δίνοντας έναν γονέα σαν αυτόν που θα δεχθεί στην επίλυση του προβλήματος το παράδειγμα που θα δοθεί παρακάτω θα έχει ως παραμέτρους, parent1 και parent2, δύο διάνυσμα με στοιχεία από το 100 έως το 199 και από το 200 έως το 299:

```
>> parent1 = [0 1 1 1 1 1 1 0];
>> parent2 = [0 1 0 1 0 1 0 1];
>> [child1, child2] = crossover(parent1, parent2, 8); % Μέγεθος των γονέων = 8.
```

Το αποτέλεσμα είναι:

	parent1	[0,1,1,1,1,1,1,0]		child1	[0,1,0,1,1,1,1,0]
	parent2	[0,1,0,1,0,1,0,1]		child2	[0,1,1,1,0,0,0,1]

Εδώ το a ήταν $a=4$ οπότε όπως φαίνεται το πρώτο παιδί είναι ίσο με τον parent2 για τα πρώτα τέσσερα στοιχεία του και ίσο με τον parent1 για τα 4 τελευταία στοιχεία του. Για το δεύτερο απόγονο (child2) ισχύει το αντίστροφο.

5. Μετάλλαξη

Η υλοποίηση αυτής της συνάρτησης είναι λίγο πιο περίπλοκη, αλλά εύκολη στην λογική της.

Η λογική, λοιπόν, που ακολουθάει η συνάρτηση είναι ο εξής:

Επειδή θέλουμε να επιλέγουμε ένα μία σειρά με πιθανότητα περίπου 0.02, αλλά κάθε φορά ένα στοιχείο της σειράς, για να κρατήσουμε τον αλγόριθμο όσο το δυνατόν πιο απλό γίνεται, διαιρέσαμε την πιθανότητα 0.02 με το μήκος της λύσης και το εφαρμόσαμε σε κάθε στοιχείο της λύσης ξεχωριστά.

Αν εν τέλει ένα στοιχείο επιλεγεί να αλλάξει (δηλαδή να υποστεί μετάλλαξη), γίνεται έλεγχος, από τον οποίο προκύπτουν 3 πιθανές διακλαδώσεις:

- Αν το στοιχείο είναι 1 γίνεται 0, χωρίς περαιτέρω πράξεις.
- Αν το στοιχείο είναι 0 και οι άσσοι της λίστας είναι 2 ή λιγότεροι, τότε το 0 γίνεται 1.
- Αν το στοιχείο είναι 0 και οι άσσοι της λίστας είναι 3 (δεν μπορούν να είναι περισσότεροι στο δικό μας παράδειγμα, σε γενικότερη περίπτωση το νούμερο

δεν είναι 3, αλλά δίνεται από τις εξόδους της `imp_vals()`, τότε το 0 γίνεται 1 και επιλέγεται ένας από τους αρχικούς άσσους τυχαία και γίνεται 0.

```

1  function mutated_child = mutation(child)
2  -     mutated_child = child;
3  -     for i=1:length(child)
4  -         randomNum = rand();
5  -         if randomNum<(0.02/length(child))
6  -             if mutated_child(i)==1
7  -                 mutated_child(i)=0;
8  -             else
9  -                 b = find(mutated_child==1);
10 -                 if length(b)==3
11 -                     indexb = randi([1 length(b)]);
12 -                     oneToZero = b(indexb);
13 -                     mutated_child(i)=1;
14 -                     mutated_child(oneToZero) = 0;
15 -                 else
16 -                     mutated_child(i)=1;
17 -                     disp(mutated_child)
18 -                 end
19 -             end
20 -         end
21 -     end
22 - end

```

Παράδειγμα:

Τρέχουμε τις εντολές

```
>> pop = gen_init_population;
```

```
>> mut_pop1 = mutation(pop(1, 1:end));
```

Έχουμε τρέξει τον αλγόριθμο της μετάλλαξης, λοιπόν, στο πρώτο στοιχείο του πληθυσμού.

Το αν έχει γίνει κάποια αλλαγή το ελέγχουμε με την εξής εντολή:

```
>> changes = find(mut_pop1 ~= pop(1, 1:end)) %Επιστρέφει σε ποιες θέσεις έχουν
```

%γίνει αλλαγές μεταξύ των στοιχείων

Το αποτέλεσμα στην πρώτη εκτέλεση:

```
>> changes = find(mut_pop1 ~= pop(1,1:end))

changes =

1x0 empty double row vector
```

Επειδή η πιθανότητα να γίνει αλλαγή σε κάποιο μέλος είναι μικρή, γράφτηκε το παρακάτω script για να εκτελέσει την συνάρτηση mutation, μέχρι να κάνει κάποια αλλαγή και να την περάσει στο νέο μέλος:

```
1 function [child] = test_mut(parent)
2     i = 0;
3     child = mutation(parent);
4     while child == parent
5
6         child = mutation(parent);
7         i = i + 1;
8     end
9     changes = find(child ~= parent)
10    fprintf('We needed %d attempts!\n', i);
11 end
```

Εκτελώ το script και στην συνέχεια παρατηρώ ότι αλλαγές έγιναν στις θέσεις 61 και 91 οπότε τυπώνω τα δύο διανύσματα από την θέση 61 έως την 91 για να δω τις διαφορές:

```
We needed 11 attempts!
>> pop(1, 61:91)

ans =

Columns 1 through 17
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

Columns 18 through 31
0 0 0 0 0 0 0 0 0 0 0 0 1 1

>> mut_pop1(61:91)

ans =

Columns 1 through 17
1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

Columns 18 through 31
0 0 0 0 0 0 0 0 0 0 0 0 1 0

>>
```

Ακόμη τυπώθηκε και στο πάνω μέρος πόσες επαναλήψεις χρειάστηκαν για να εφαρμοστεί μετάλλαξη στο μέλος που δόθηκε ως argument.

6. Επιλογή – Αναπαραγωγή

Εδώ επειδή προτάθηκε από την θεωρία να γίνεται επιλογή όλων των μελών με κάποια πιθανότητα, απλά τα μέλη με μεγαλύτερη τιμή στην αξιολόγηση να έχουν μεγαλύτερη πιθανότητα να επιλεγούν, χρησιμοποιήθηκε μία συνάρτηση που προσεγγίζει την επιλογή σε μορφή ρουλέτας. Η έξοδος της συνάρτησης δεν είναι η θέση των 2 μελών που επιλέγονται, αλλά τα μέλος-διανύσματα τα ίδια για να απλοποιεί την διαδικασία σε μετέπειτα στάδιο επίλυσης του προβλήματος.

```

1 function [parent1, parent2] = myRouletteSel(popArray, fitList)
2     if nargin<2
3         fitList = fit_co(popArray);
4     end
5
6     tot_fitness = sum(fitList);
7     prob_list = fitList./tot_fitness;
8
9     parentIndex1 = randsample(1:length(prob_list), 1,true, prob_list);
10    parentIndex2 = randsample(1:length(prob_list), 1,true, prob_list);
11    while (parentIndex1==parentIndex2)
12        parentIndex2 = randsample(1:length(prob_list), 1,true, prob_list);
13    end
14
15    parent1 = popArray(parentIndex1, 1:end);
16    parent2 = popArray(parentIndex2, 1:end);
17
18 end
19

```

Δίνεται η λίστα με την αξιολόγηση κάθε λύσης ως παράμετρος τα συνάρτησης για να μην δαπανείται χρόνος υπολογισμού εσωτερικά. Αυτό βελτιώνει δραματικά τον χρόνο εκτέλεσης του προγράμματος όταν καλείται μέσω της τελικής συνάρτησης, καθώς η συγκεκριμένη συνάρτηση (myRouletteSel) θα εκτελεστεί χιλιάδες φορές μέχρι τον τερματισμό και την εύρεση της λύσης. Αν δεν δοθεί η λίστα με τις αξιολογήσεις η συνάρτηση από default την υπολογίζει. Στην συνέχεια παίρνουμε το άθροισμα όλων των αξιολογήσεων και δημιουργούμε μία κανονικοποιημένη λίστα των αξιολογήσεων διαιρώντας όλες τις αξιολογήσεις με το συνολικό άθροισμα.

Αυτή η κανονικοποιημένη λίστα δίνει την πιθανότητα επιλογής της κάθε λύσης.

Στο τέλος επιστρέφει μέσω των μεταβλητών parent1 και parent2 τα επιλεγμένα μέλη ως γονείς.

Παράδειγμα

```
>> [p1, p2] = gen_init_population(32);
```

Εκτελώντας αυτή την εντολή, αν δούμε δεξιά στην λίστα των μεταβλητών, έχουμε:

p1	1x100 double
p2	1x100 double
pop	32x100 double

7. Παραγωγή Νέας Γενιάς

Η συνάρτηση που θα παράγει την νέα γενιά είναι η παρακάτω:

```
1 function next_gen = nextGeneration(population, sizeOfPopulation, sizes_list, values_list, solSize, maxSize)
2 -
3 -     if nargin<5
4 -         if nargin<2
5 -             sizeOfPopulation = size(population, 1);
6 -         end
7 -         [sizes_list, values_list, solSize, maxSize, ~] = imp_vals();
8 -     end
9 -
10 -     fit_list = fit_co(population, sizes_list, values_list, solSize, maxSize);
11 -
12 -     next_gen = [];
13 -
14 -     a = pass_n_toNextGen(population, fit_list);
15 -
16 -     next_gen = [a; next_gen];
17 -
18 -     while size(next_gen, 1)<sizeOfPopulation
19 -         [parent1, parent2] = myRouletteSel(population, fit_list);
20 -
21 -         if rand() < 0.45
22 -             temp_child1 = parent1;
23 -             temp_child2 = parent2;
24 -         else
25 -             [temp_child1, temp_child2] = crossover(parent1, parent2, solSize);
26 -         end
27 -         temp_child1 = mutation(temp_child1);
28 -         temp_child2 = mutation(temp_child2);
29 -
30 -         next_gen = [next_gen ; temp_child1; temp_child2];
31 -         if size(next_gen, 1)>=sizeOfPopulation
32 -             next_gen = unique(next_gen, 'rows');
33 -         end
34 -     end
35 - end
```

Η συνάρτηση `nextGeneration`, δέχεται ως παραμέτρους των πληθυσμό, το μέγεθος του (default να υπολογίσει το μέγεθος από μόνο του), την λίστα με τα μεγέθη των αντικειμένων, την λίστα με τις αξίες των αντικειμένων, το μήκος κάθε λύσης, το `maxSize` το οποίο δείχνει το μέγεθος του σακιδίου.

Αρχικά ορίζουμε έναν κενό πίνακα στην μεταβλητή `next_gen` η οποία σε κάθε επανάληψη θα αποκτήει ένα νέο μέλος σε κάθε επανάληψη του βρόχου `while` μέχρι να γίνει ίση σε μέγεθος με τον αρχικό πληθυσμό και υπολογίζουμε τον πίνακα με την αξιολόγηση του πληθυσμού.

Στην συνέχεια καλούμε μία συνάρτηση που θα αναλύσουμε στην συνέχεια. Αυτή η συνάρτηση επιστρέφει τις η καλύτερες λύσεις (στην προκειμένη περίπτωση 4). Στην συνέχεια, τις περνάει άμεσα στην επόμενη γενιά.

Στον βρόχο, αρχικά, επιλέγονται 2 γονείς με βάση την προηγούμενη συνάρτηση επιλογής που αναλύθηκε. Στην συνέχεια με πιθανότητα 45% περνάμε τους γονείς αυτούσιους ή, εναλλακτικά, εκτελούμε την συνάρτηση `crossover`.

Αφού παραχθεί τα 2 μέλη που θα περάσουν στην επόμενη γενιά, εκτελούμε πάνω σε αυτά την συνάρτηση `mutation()`.

Προσθέτουμε το νέο μέλος στον πίνακα της επόμενης γενιάς.

Πριν κλείσει ο βρόχος κρατάμε όλα τα μέλη του πληθυσμού που είναι διαφορετικά μεταξύ τους.

Παράδειγμα

```
>> pop = gen_init_population(32);
```

```
>> next_gen = nextGeneration(pop);
```

Επειδή, όμως εδώ απλά θα έχουμε δύο διαφορετικούς αραιούς πίνακες 32x100 που δεν θα μπορούμε να τους ελέγξουμε εύκολα.

Οπότε τρέχουμε και τις εντολές:

```
>> fit_pop = mean(fit_co(pop))
```

```
>> fit_next_gen = mean(fit_co(next_gen))
```

Τα αποτελέσματα:

Name ▲	Value
fit_next_gen	459.1250
fit_pop	243.8750
next_gen	32x100 double
pop	32x100 double

Φαίνεται ότι ο μέσος όρος των αξιολογήσεων της πρώτης γενιάς είναι χειρότερος από αυτόν της δεύτερης. Οπότε υπάρχει άμεση βελτίωση.

Όσον αφορά την συνάρτηση `pass_n_toNextGen()`, ο κώδικας της δίνεται παρακάτω:

```

1  function [a] = pass_n_toNextGen(population, numOfpass, fit)
2  -     if nargin<2
3  -         fit = fit_co(population);
4  -     end
5
6  -     a = nan(numOfpass, size(population,2));
7
8  -     sortedFit = sort(fit);
9  -     sortedFit = sortedFit(end-numOfpass+1:end);
10 -     for i=1:numOfpass
11 -         index = find(fit==sortedFit(i));
12 -         temp = population(index(1), 1:end);
13 -         a(i, 1:end) = temp;
14 -     end
15 - end
16 - end

```

Στην ουσία, αρχικά, ταξινομεί την λίστα με τις αξιολογήσεις και στην συνέχεια ανάλογα με την θέση που έχουν στην αρχική λίστα οι η καλύτερες λύσεις επιστρέφει τα μέλη του πληθυσμού με αυτές τις αξιολογήσεις.

8. Επίλυση – Κριτήριο Τερματισμού

Η τελική συνάρτηση που θα καλείται και θα επιλύει το πρόβλημα θα είναι η παρακάτω:


```

1 function [items] = solveKnapsack(filename)
2     if nargin<1
3         [sizes_list, values_list, solSize, maxSize, maxItems] = imp_vals();
4     else
5         [sizes_list, values_list, solSize, maxSize, maxItems] = imp_vals(filename);
6     end
7     population = gen_init_population(32, solSize, maxItems);
8     iterations = ceil(0.45*maxItems*maxItems*solSize);
9     avg_fitness = nan(iterations, 1);
10    max_fitness = nan(iterations, 1);
11
12
13    for i=1:iterations
14        max_fitness(i) = max(fit_co(population, sizes_list, values_list, solSize, maxSize));
15        avg_fitness(i) = mean(nonzeros(fit_co(population,sizes_list, values_list, solSize, maxSize)));
16        pop_temp = nextGeneration(population, 64, sizes_list, values_list, solSize, maxSize);
17        population = pop_temp;
18    end
19
20    figure
21    plot(1:length(avg_fitness), avg_fitness);
22    title("Average Fitness");
23    figure
24    plot(1:length(max_fitness), max_fitness);
25    title("Max Fitness");
26    %}
27    list = fit_co(population, sizes_list, values_list, solSize, maxSize);
28    IndOfMax = find(max(list)==list);
29    fit_sol = population(IndOfMax(1), 1:end);
30    items = find(fit_sol==1);
31 end

```

Στις παραμέτρους της συνάρτησης μπορούμε προαιρετικά να βάλουμε το αρχείο σαν path από το οποίο θα αντλήσει τα στοιχεία. Αν δεν δοθεί το αρχείο, ως προεπιλογή θα αντλήσει η συνάρτηση αρχεία από το **knapsack.xls** .

Από εκεί και μετά, ξεκινάει η πραγματική λειτουργία της συνάρτησης.

Αρχικά, παράγεται ένας τυχαίος αρχικός πληθυσμός με 32 μέλη και γίνεται ένας υπολογισμός για το πόσες επαναλήψεις θα χρειαστούν για να τερματίσει ο αλγόριθμος. Μετά από πολλαπλές απόπειρες κατέληξα ότι η πράξη είναι η παρακάτω:

$$[0.45 \cdot (\text{μεγιστος αριθμός αντικειμένων})^2 \cdot (\text{μήκος υποψήφιων λύσεων})]$$

Δίνει ικανοποιητικό χρόνο τόσο για την εύρεση λύσης για τις τιμές που δίνονται στο αρχικό αρχείο που δόθηκε, αλλά και για σακίδιο με μεγαλύτερο χώρο και για περισσότερα υποψήφια αντικείμενα.

Στην συνέχεια αρχικοποιούνται τα διανύσματα avg_fitness και max_fitness που έχουν μήκος ίσο με τις επαναλήψεις, η ύπαρξη τους δεν έχει χρησιμότητα, απλά οπτικοποιεί την διαδικασία εύρεσης λύσης δίνοντας τον μέσο όρο αξιολόγησης κάθε γενιάς και την αξιολόγηση της καλύτερης λύσης για κάθε γενιά.

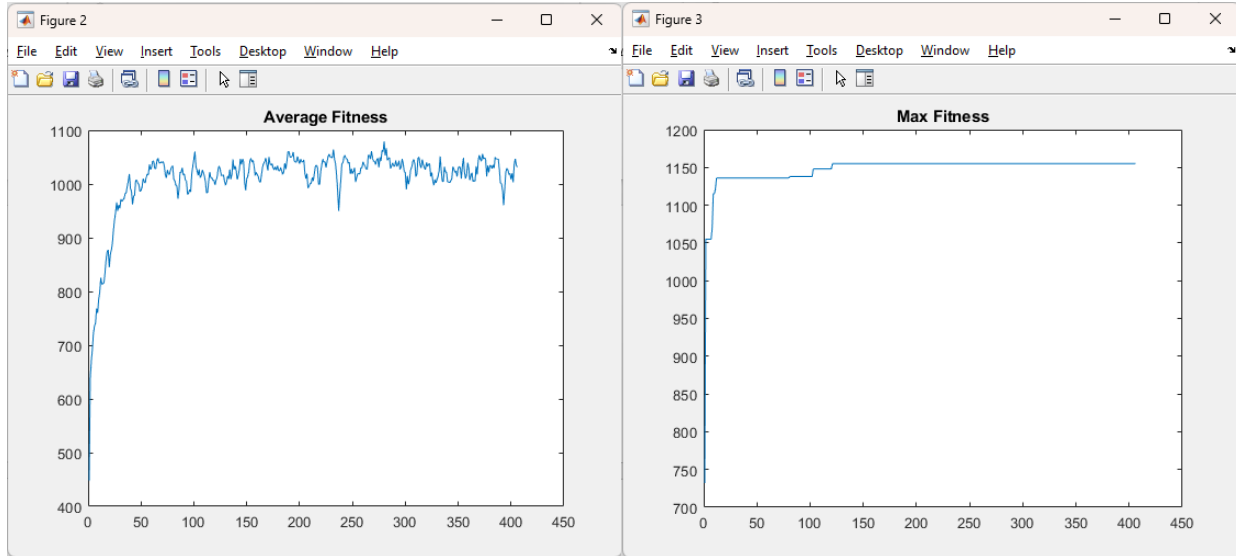
Στην συνέχεια, μέσω της for επανάληψης παράγει την κάθε γενιά. Στο τέλος παράγονται 2 διαγράμματα με την μέση αξιολόγηση κάθε γενιάς και την καλύτερη λύση κάθε γενιάς, για να δει ο χρήστης πόσο γρήγορα σύγκλινε στην λύση ο αλγόριθμος.

Τέλος στις γραμμές 27-29 υπολογίζεται και επιστρέφεται μέσω της `fit_sol` η καλύτερη λύση της τελευταίας γενιάς, ενώ στην γραμμή 30 εισάγεται στην μεταβλητή εξόδου το διάνυσμα που δείχνει ποια αντικείμενα επιλέχθηκαν.

Παράδειγμα:

```
>> a = solveKnapsack()
```

Εμφανίζονται δύο παράθυρα με τα διαγράμματα:



(Δεν είναι πάντοτε τόσο γρήγορη η εύρεση της καλύτερης λύσης όσο φαίνεται στο δεξί διάγραμμα)

Επίσης στην μεταβλητή `a` έχει εκχωρηθεί η λύση του προβλήματος και μπορούμε να δούμε τα αντικείμενα που επιλέχθηκαν:

```
>> a = solveKnapsack()

a =

    58    93    97
```

Γ. ΣΗΜΕΙΩΣΕΙΣ

Η αρχική έκδοση της λύσης είχε χρόνο εκτέλεσης περίπου 1 ώρα για τις παραμέτρους που δινόταν στο αρχείο. Μέσω του profiler του MATLAB ανακαλύφθηκε ότι ο λόγος που γινόταν αυτό ήταν γιατί αρχικά υπήρχαν 3 παράγοντες που επιβράδιναν εξαιρετικά το πρόγραμμα:

- 1) Ότι υπήρχαν πίνακες που άλλαζαν μέγεθος μέσα σε βρόχους

- 2) Η συνάρτηση `imp_vals()` εκτελούνταν κάθε φορά που εκτελούνταν η συνάρτηση `fit_co()`, αλλά και στην `gen_init_population()`,
- 3) Και η συνάρτηση `fit_co()` που εκτελούνταν επίσης κάθε φορά που εκτελούνταν η `myRouletteSel()` και η `pass_n_toNextGen()`, αλλά και η `nextGeneration()`.

Στο συγκεκριμένο πρόβλημα που γίνονται παράγονται περίπου 380, λοιπόν, οι συνάρτηση `imp_vals()` η οποία είναι και η πιο βαριά υπολογιστικά, σε χρόνο, και χρειαζόταν περίπου 0.7s (στον profiler οι εκτελέσεις γίνονται πιο αργά), εκτελούνταν εκατοντάδες χιλιάδες φορές και επιπρόσθετα τρομερά το πρόγραμμα.

Διαδικασία βελτίωσης του κώδικα:

- 1) Ξεκίνησα επιλύοντας το πρόβλημα των πινάκων. Αντί να προσθέτω αντικείμενα στο τέλος, έκανα `preallocation` τους πίνακες σχεδόν σε όλες τις επιμέρους συναρτήσεις εκτός από την συνάρτηση `nextGeneration()` όπου αποδείχθηκε ιδιαίτερα περίπλοκο να το αποφύγω.
- 2) Για να μην χρειάζεται να εκτελείται μέσα στην `fit_co()` η `imp_vals()`, έδινά τις εξόδους τις `imp_vals()`, η οποία τρέχει εκτός της `fit_co()` ούτως ή άλλως, σαν παραμέτρους εισόδου στην `fit_co()`.
- 3) Για την μείωση των εκτελέσεων της `fit_co()` αφού σε κάθε γενιά της `nextGeneration()` υπήρχε ήδη ένας υπολογισμός για την αξιολόγηση της γενιάς, περνούσα την 'δη υπάρχουσα αξιολόγηση ως παράμετρο εισόδου στις συναρτήσεις `myRouletteSel()` και `pass_n_toNextGen()`.

Το αποτέλεσμα κάθε βελτίωσης σε χρόνο ήταν το παρακάτω:

- 1) Από περίπου 50-60 λεπτά εκτέλεσης του κώδικα, ο χρόνος μειώθηκε σε 25-30 λεπτά.
- 2) Ο χρόνος εκτέλεσης από τα 25-30 λεπτά μειώθηκε στα 1-2 λεπτά
- 3) Τέλος η τελευταία διόρθωση μείωσε τον χρόνο ακόμα περισσότερο με τελικό χρόνο εκτέλεσης τα 3 – 4,5 δευτερόλεπτα.

Τέλος για το παρόν πρόβλημα που δόθηκε το ποσοστό επιτυχίας εύρεση της καλύτερης λύσης είναι της τάξης 91%-92%, ενώ το ποσοστό εύρεσης της πρώτης ή της δεύτερης καλύτερης λύσης είναι 100% (ή τουλάχιστον 99%).

Αυτά τα ποσοστά φαίνονται μετά από 350 εκτελέσεις του κώδικα στο επόμενο διάγραμμα.

